KNAPSACK METAHEURISTIC OPTIMISATION

Table of Contents

Introduction

The 0/1 Knapsack Problem is a classical NP-hard optimisation problem in which a set of items, each with an associated weight and value, must be selected to maximise total value without exceeding a predefined capacity constraint. Due to its computational complexity, exact methods become inefficient for larger instances, making metaheuristic approaches more suitable.

This report compares the performance of two metaheuristic algorithms: a Genetic Algorithm (GA) and Iterated Local Search (ILS). The GA is a population-based method inspired by natural evolution, while ILS is a trajectory-based approach that iteratively refines a single solution. The study evaluates their performance across multiple benchmark instances in terms of solution quality and runtime, with statistical validation performed using the Wilcoxon signed-rank test.

To ensure reliable results, each algorithm was executed using ten independent random seeds: 42, 3, 2005, 21, 515, 7, 99, 123, 777, and 2026. This was carried out across ten benchmark instances from the low-dimensional knapsack dataset, resulting in a total of 200 runs. For each instance, the average runtime and best solution quality were recorded across all seeds. Subsequently, a one-tailed Wilcoxon Signed-Rank Test was applied to determine if there is a statistically significant performance difference between the two algorithms.

1. Genetic Algorithm(GA)

1.1. Solution Representation

Each individual is a binary chromosome of length n. Gene $i = 1$ means item i is selected; 0 means excluded. This is the standard encoding for 0/1 knapsack (Khuri, Bäck and Heitkötter, 1994).

```
private final int[] selection;
private double totalValue;
private double totalWeight;
```

Figure 1.1: Solution Representation (code snippet from KnapsackSolution.java)

1.2. Population Initialisation

Population size $P = 100$. Each individual is generated by random binary initialisation with a feasibility check: items are shuffled (Fisher-Yates), then each is included with 50% probability if it fits. This guarantees all initial solutions are feasible and diverse.

```
for (int idx : indices) {
    if (rng.nextDouble() < 0.5) {
        if (sol.getTotalWeight() + instance.getWeight(idx) <= instance.getCapacity()) {
            sol.setItem(idx, value: 1);
        }
    }
}
```

Figure 1.2: Population Initialisation (code snippet from KnapsackSolution.generateRandom())

1.3. Fitness Function

Fitness equals the total value of selected items, evaluated only if the total weight does not exceed W. The repair-and-fill operator (Section 1.6) guarantees that all population members are always feasible, so a penalty function is not required.

1.4. Selection

Two individuals are drawn uniformly at random (with replacement); the fitter one is selected as a parent. Tournament size $k = 2$ provides low selection pressure, preserving diversity while still favouring higher-quality solutions (Goldberg and Deb, 1991).

```
KnapsackSolution best = population[rng.nextInt(POPULATION_SIZE)];
for (int i = 1; i < TOURNAMENT_SIZE; i++) {
    KnapsackSolution candidate = population[rng.nextInt(POPULATION_SIZE)];
    if (candidate.getTotalValue() > best.getTotalValue()) {
        best = candidate;
    }
}
return new KnapsackSolution(best);
```

Figure 1.3: Binary Tournament (code snippet from GeneticAlgorithm.tournamentSelect())

1.5. Crossover

With probability $P_c = 0.85$, two parents produce two offspring via uniform crossover: each gene position is independently inherited from parent1 or parent2 with equal probability (0.5). If crossover is not applied, the parents are cloned. Uniform crossover avoids positional bias, which is appropriate when item indices are arbitrary.

```
for (int i = 0; i < n; i++) {
    if (rng.nextBoolean()) {
        child1.setItem(i, p1.getItem(i));
        child2.setItem(i, p2.getItem(i));
    } else {
        child1.setItem(i, p2.getItem(i));
        child2.setItem(i, p1.getItem(i));
    }
}
return new KnapsackSolution[]{child1, child2};
```

Figure 1.4: Uniform Crossover (code snippet from GeneticAlgorithm.uniformCrossover())

1.6. Mutation

Each gene is flipped with probability $1/n$, giving an expected one flip per chromosome. This is the theoretically recommended rate for binary GAs (Bäck, 1996); it introduces diversity without destroying structures built by crossover.

```
double mutationRate = 1.0 / n;
for (int i = 0; i < n; i++) {
    if (rng.nextDouble() < mutationRate) {
        solution.flipItem(i);
    }
}
```

Figure 1.5: Bit Flip (code snippet from GeneticAlgorithm.bitFlipMutate())

1.7. Constraint Handling

After mutation, every offspring passes through a two-phase operator:

- **Repair phase:** If total weight > W, selected items are sorted ascending by value/weight ratio (least efficient first) and removed one by one until feasible.
- **Fill phase:** Unselected items are sorted descending by value/weight ratio and greedily added if they fit. This recovers value lost during repair.

```
Arrays.sort(selected, fromIndex: 0, count, (a, b) -> Double.compare(
        safeRatio(instance.getValue(a), instance.getWeight(a)),
        safeRatio(instance.getValue(b), instance.getWeight(b))
));

for (int i = 0; i < count && !solution.isFeasible(); i++) {
    solution.setItem(selected[i], value: 0);
}
```

Figure 1.c: Repair and Flip Operator (code snippet from GeneticAlgorithm.repairAndFill())

1.8. Replacement

A full new population of 100 individuals is created each generation. The top 2 fittest individuals from the current generation are copied unchanged into the next generation (elitism), guaranteeing monotonic improvement of the best solution (De Jong, 1975). The remaining 98 slots are filled by offspring from tournament selection, crossover, mutation, and repair-and-fill.

1.6. Termination

The GA runs for MAX_GENERATIONS = 1000 generations. The global best solution found across all generations is tracked and returned.

2. Iterated Local Search (ILS)

ILS is a trajectory-based metaheuristic that works on a single solution. It repeatedly applies a local search to reach a local optimum, then perturbs that solution to escape the basin of attraction, and accepts or rejects the new solution based on an adaptive criterion (Lourenço, Martin and Stützle, 2003).

2.1. Initial Solution

Items are sorted in descending order of value/weight ratio, with random tie-breaking (the comparator returns ±1 randomly for equal ratios). Items are then added greedily if they fit. This yields high-quality starting points with seed-dependent diversity.

```
java.util.Arrays.sort(indices, (a, b) -> {
    double ratioA = safeRatio(instance.getValue(a), instance.getWeight(a));
    double ratioB = safeRatio(instance.getValue(b), instance.getWeight(b));
    int cmp = Double.compare(ratioB, ratioA);
    return cmp != 0 ? cmp : (rng.nextBoolean() ? 1 : -1);
});

// Greedily add items in sorted order if they fit
for (int idx : indices) {
    if (sol.getTotalWeight() + instance.getWeight(idx) <= instance.getCapacity()) {
        sol.setItem(idx, value: 1);
    }
}
```

Figure 2.1: Greedy Randomised Construction (code snippet from KnapsackSolution.generateGreedyRandom())

2.2. Local Search

The local search runs until no improvement is found (a local optimum). It uses first-improvement and two neighbourhoods:

- **Phase 1 - 1-flip:** All n items are tried in random order. Toggling an item that improves feasibility and value is accepted immediately; the phase restarts.
- **Phase 2 - Swap (1-1 exchange):** If no 1-flip improves, all pairs (i, j) with $x_i = 1, x_j = 0$ are tried in random order. Removing i and adding j is accepted if it improves value and stays feasible.

After reaching a local optimum, a greedy fill adds any unselected items (descending v/w) that fit in the remaining capacity.

```java
int[] order = getShuffledIndices(n);
for (int idx : order) {
    double oldValue = current.getTotalValue();
    current.flipItem(idx);

    if (current.isFeasible() && current.getTotalValue() > oldValue) {
        improved = true;
        break;
    } else {
        current.flipItem(idx);
    }
}
```

Figure 2.2: Combined 1-Flip and Swap Neighbourhoods (code snipper from IteratedLocalSearch.localSearch())

2.3. Perturbation

To escape the local optimum, the algorithm flips $k = 3$ randomly selected bits (without replacement). If the perturbation makes the solution infeasible, a repair operator removes selected items in ascending v/w order until feasibility is restored. The perturbation strength is capped at $n/2$ for very small instances (e.g., $n = 4 \rightarrow k = 2$).

```java
int flips = Math.min(perturbationStrength, n);
int[] indices = getShuffledIndices(n);

// Flip 'flips' randomly chosen bits
for (int i = 0; i < flips; i++) {
    perturbed.flipItem(indices[i]);
}

// Repair if the perturbation made the solution infeasible
repair(perturbed);
```

Figure 2.3: Random k-Bit-Flip (code snippet from IteratedLocalSearch.perturbation())

2.4. Acceptance Criterion – SA-Style Probabilistic

Improving or equal solutions are always accepted.

Worse solutions are accepted with probability $exp\ (-\delta/(0.05 \cdot \text{bestVal}))$, where δ = current.value − candidate.value. This simulated annealing-like mechanism allows occasional uphill moves, scaled to the magnitude of the best solution found so far.

```
if (candidate.getTotalValue() >= current.getTotalValue()) {
    return candidate;
}

// Probabilistic acceptance of worse solutions to maintain diversity
double delta = current.getTotalValue() - candidate.getTotalValue();
double bestVal = best.getTotalValue();
double threshold = bestVal > 0 ? Math.exp(-delta / (0.05 * bestVal)) : 0.1;

if (rng.nextDouble() < threshold) {
    return candidate;
}

return current;
```

Figure 2.4: SA style probabilistic (code snippet from IteratedLocalSearch.acceptanceCriterion())

2.5. Termination

ILS runs for MAX_ITERATIONS = 5000 iterations. Each iteration consists of one perturbation followed by a full local search to the next local optimum. The overall best solution encountered is returned.

3. Experimental Results

Both algorithms were executed across 10 seeds (42, 3, 2005, 21, 515, 7, 99, 123, 777, 2026) on all 10 benchmark instances, resulting in a total of 200 algorithm executions. All runs used the combined Knapsack.jar executable, which displays the results of ILS and GA side by side. The seed is provided interactively at runtime. The results below present the output for a single representative seed (seed 42), followed by aggregated averages across all 10 seeds.

Problem Instance	Algorithm	Seed Value	Best Solution	Known Optimum	Runtime (seconds)
f1_l-d_kp_10_269	GA	42	295	295	0.251002
	ILS	42	295	295	0.080521
f2_l-d_kp_20_878	GA	42	1024	1024	0.145958
	ILS	42	1024	1024	0.105978
f3_l-d_kp_4_20	GA	42	35	35	0.096514
	ILS	42	35	35	0.006898
f4_l-d_kp_4_11	GA	42	23	23	0.071422
	ILS	42	23	23	0.010317
f5_l-d_kp_15_375	GA	42	481.0694	481.0694	0.144434
	ILS	42	481.0694	481.0694	0.047216
f6_l-d_kp_10_60	GA	42	52	52	0.069877
	ILS	42	52	52	0.034124
f7_l-d_kp_7_50	GA	42	107	107	0.072861
	ILS	42	107	107	0.014083
f8_l-d_kp_23_10000	GA	42	9767	9767	0.185040
	ILS	42	9767	9767	0.079056
f9_l-d_kp_5_80	GA	42	130	130	0.054191
	ILS	42	130	130	0.010227

f10_l-d_kp_20_879	GA	42	1025	1025	0.131130
	ILS	42	1025	1025	0.060774

Table 1: Results for a single representative seed (42)

3.1. Solution Quality

Instance	n	W	ILS Best Solution	GA Best Solution	Known Optimum	Gap (%)
f1_l-d_kp_10_269	10	269	295	295	295	0.00%
f2_l-d_kp_20_878	20	878	1024	1024	1024	0.00%
f3_l-d_kp_4_20	4	20	35	35	35	0.00%
f4_l-d_kp_4_11	4	11	23	23	23	0.00%
f5_l-d_kp_15_375	15	375	481.0694	481.0694	481.0694	0.00%
f6_l-d_kp_10_60	10	60	52	52	52	0.00%
f7_l-d_kp_7_50	7	50	107	107	107	0.00%
f8_l-d_kp_23_10000	23	10000	9767	9767	9767	0.00%
f9_l-d_kp_5_80	5	80	130	130	130	0.00%
f10_l-d_kp_20_879	20	879	1025	1025	1025	0.00%

Table 2: Best solution found per algorithm per instance (identical across all 10 seeds)

Both algorithms achieved the known optimal solution on every instance across every seed, a total of 200 out of 200 runs reaching the optimum. The optimality gap is 0.00% in all cases. This result is seed-independent: whether using seed 3, 42, 777, or 2026, both algorithms reliably locate the global optimum.

3.2. Runtime Results

Instance	n	ILS Avg Runtime (s)	GA Avg Runtime (s)	GA/ILS Ratio
f1_l-d_kp_10_269	10	0.080521	0.251002	3.12×
f2_l-d_kp_20_878	20	0.105978	0.145958	1.38×
f3_l-d_kp_4_20	4	0.006898	0.096514	13.99×
f4_l-d_kp_4_11	4	0.010317	0.071422	6.92×

f5_l-d_kp_15_375	15	0.047216	0.144434	3.06×
f6_l-d_kp_10_60	10	0.034124	0.069877	2.05×
f7_l-d_kp_7_50	7	0.014083	0.072861	5.17×
f8_l-d_kp_23_10000	23	0.079056	0.185040	2.34×
f9_l-d_kp_5_80	5	0.010227	0.054191	5.30×
f10_l-d_kp_20_879	20	0.060774	0.131130	2.16×

Table 3: Average runtime (seconds) across 10 seeds per instance, with GA/ILS speed ratio

The GA/ILS ratio compares the average runtimes of the Genetic Algorithm (GA) and Iterated Local Search (ILS). A value over 1 shows that the GA is slower, with higher values indicating a larger performance gap. In total, across 100 runs (10 seeds × 10 instances), ILS averaged 0.044919 seconds per instance, while the GA averaged 0.122243 seconds per instance, making the GA about 2.72 times slower. The ratio varies by instance: the smallest discrepancy is 1.38× for instance f2 (n = 20), and the largest is 13.99× for instance f3 (n = 4). Overall, the GA is consistently slower than ILS across all instances.

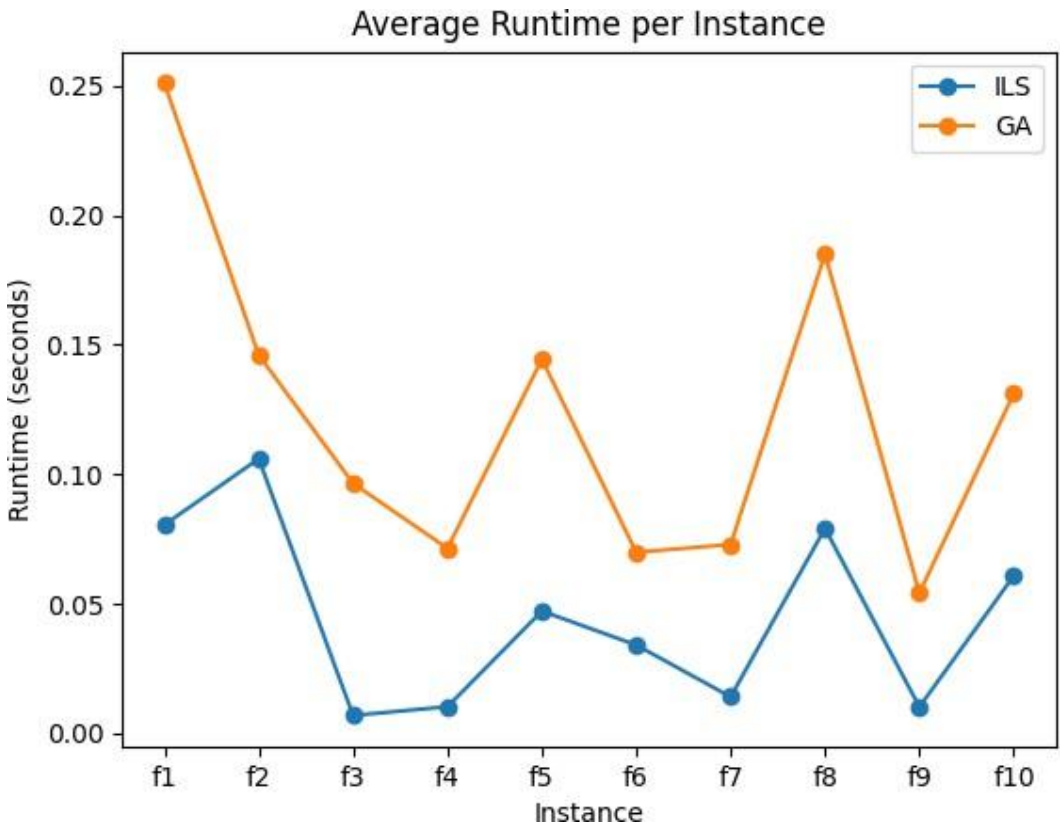

Figure 3.2: Average runtime (seconds) per instance for GA and ILS across 10 seeds

Figure 3.2 visually confirms the numerical results presented in Table 3. For every instance, the runtime of the Genetic Algorithm is consistently higher than that of Iterated Local Search. The gap is particularly pronounced for smaller instances (f3, f4, f9), where the overhead of maintaining a population dominates the computation. As the instance size increases, the gap narrows, but ILS remains consistently faster.

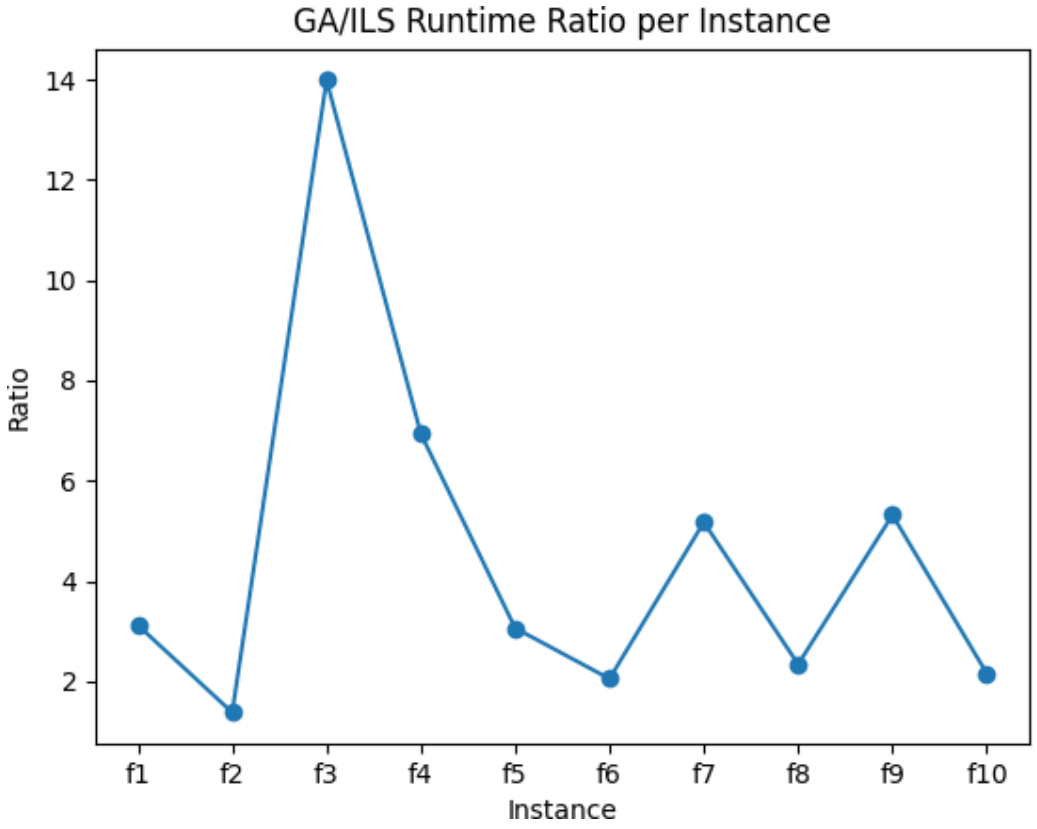

Figure 3.3: GA/ILS runtime ratio per instance.

Figure 3.3 highlights the magnitude of the runtime difference between the two algorithms. All ratios are greater than 1, confirming that the GA is slower in every case. The ratio peaks at nearly 14× for instance f3, demonstrating extreme inefficiency of GA on very small problems. The decreasing trend as n increases suggests that the relative overhead of GA becomes less dominant for larger problem sizes.

3.3. Seed-to-Seed Variability in Runtime

Seed	ILS Total (s)	GA Total (s)	GA/ILS Ratio
42	0.433400	1.294385	2.99×
3	0.439626	1.257229	2.86×

2005	0.543286	1.169879	2.15×
21	0.438975	1.210650	2.76×
515	0.431182	1.200570	2.78×
7	0.445633	1.132387	2.54×
99	0.395184	1.196754	3.03×
123	0.464947	1.320331	2.84×
777	0.433534	1.141388	2.63×
2026	0.466182	1.300719	2.79×

Table 4: Per-seed total runtime (summed across all 10 instances)

GA total runtimes range from 1.132 s (seed 7) to 1.320 s (seed 123), with a low variation of 1.17× due to its fixed structure of 1,000 generations and 100 individuals, making runtime largely independent of the seed. In contrast, ILS runtimes range from 0.395 s (seed 99) to 0.543 s (seed 2005), showing a variation of 1.37×. This is because ILS's runtime depends on the speed of local search convergence, which varies with the perturbation directions and item orderings for each seed. Faster convergence occurs with seeds generating perturbations near local optima, demonstrating that ILS adapts
its runtime to the problem's demands.

4. Statistical Analysis

4.1. Purpose and Hypotheses

The Wilcoxon Signed-Rank Test is a non-parametric statistical test for paired samples that assesses whether there is a systematic difference in their medians. It does not require any assumptions regarding the data distribution (unlike the paired t-test), which makes it suitable for benchmarking algorithms where the result distributions might not follow a normal pattern (Derrac et al., 2011).

In this study, the test is applied to the best solution values per instance, averaged over 10 seeds.

- **Null Hypothesis (H_0):** The median best solution values of GA and ILS are equivalent across all ten problem instances. Neither algorithm performs significantly better than the other.
- **Alternative Hypothesis (H_1):** One algorithm produces significantly better solutions than the other (one-tailed test, significance level $\alpha = 0.05$).

4.2. Test Data and Procedure

Across all 10 instances and 10 seeds, both algorithms consistently achieved the known optimal solution. As a result, the average best solution values for GA and ILS are identical for every instance. The differences d_i = GA_avg_best_i − ILS_avg_best_i are therefore all zero:

Instance	GA Avg Best	ILS Avg Best	d_i = GA − ILS	$\lvert d_i \rvert$	Rank	Signed Rank
f1_l-d_kp_10_269	295	295	0	0	—	—
f2_l-d_kp_20_878	1024	1024	0	0	—	—
f3_l-d_kp_4_20	35	35	0	0	—	—
f4_l-d_kp_4_11	23	23	0	0	—	—
f5_l-d_kp_15_375	481.0694	481.0694	0	0	—	—
f6_l-d_kp_10_60	52	52	0	0	—	—
f7_l-d_kp_7_50	107	107	0	0	—	—

f8_l-d_kp_23_10000	9767	9767	0	0	—	—
f9_l-d_kp_5_80	130	130	0	0	—	—
f10_l-d_kp_20_879	1025	1025	0	0	—	—

Table 5: Solution quality differences (GA avg best – ILS avg best)

According to the Wilcoxon procedure (Conover, 1999), all zero differences are removed before ranking. After removal, no observations remain (N = 0), meaning that no ranks can be assigned and the test statistic cannot be computed.

4.3. Results and Interpretation

According to the Wilcoxon procedure (Conover, 1999), all zero differences are excluded before ranking. After removal, no observations remain (N = 0), meaning that no ranks can be assigned and the test statistic W cannot be computed. This is a degenerate outcome of the test, not a valid statistical result.

It is therefore incorrect to state that we 'fail to reject H0' in any meaningful statistical sense. The test simply cannot be applied: with all differences equal to zero, there is no evidence from which to draw any inference about solution quality. The correct conclusion is that the Wilcoxon test on solution quality is inapplicable for these benchmark instances, because both algorithms achieved identical optimal results on every run.

This outcome does not imply that the algorithms are equivalent in general. It reflects that the selected benchmark instances are insufficiently challenging to discriminate between the two approaches in terms of solution quality. For larger or harder instances (e.g., n > 100), differences in best-found values are likely to emerge, enabling a meaningful statistical comparison. As solution quality cannot be statistically compared here, runtime becomes the sole dimension for differentiation and is examined in detail in Section 5.

5. Critical Analysis

5.1. Solution Quality: Both Algorithms Reach the Optimum

Both algorithms achieved a 0.00% optimality gap across all 200 runs (10 seeds × 10 instances × 2 algorithms). This outcome is consistent across widely different seeds such as 3, 7, 777, and 2026, indicating that the results are not dependent on any particular random initialisation. For the low-dimensional benchmark considered here (4 to 23 items), the search space ranges from $2^4 = 16$ to approximately $2^{23} \approx 8.4$ million possible subsets. The GA performs 100,000 evaluations per instance, while ILS executes 5,000 local search iterations. Both are more than sufficient to reliably locate the optimal solution at this scale, which explains the uniform 0% gap observed.

Since solution quality is identical across all instances and seeds, the Wilcoxon test applied to solution values (Section 4) becomes degenerate. As a result, runtime remains the only meaningful dimension along which the algorithms can be compared, and it is examined in detail below.

5.2. Runtime Analysis

ILS outperforms the GA in runtime across all instances and seeds. In all 100 paired comparisons, ILS completes faster than GA. The average GA/ILS speed ratio is 2.72×, with values ranging from 1.38× for instance f2 (20 items) to 13.99× for instance f3 (4 items). Table 6 summarises the per-instance averages.

Instance	n	ILS Avg (s)	GA Avg (s)	d_i = GA − ILS	GA/ILS Ratio
f1_l-d_kp_10_269	10	0.0805	0.2510	0.1705	3.12×
f2_l-d_kp_20_878	20	0.1060	0.1460	0.0400	1.38×
f3_l-d_kp_4_20	4	0.0069	0.0965	0.0896	13.99×
f4_l-d_kp_4_11	4	0.0103	0.0714	0.0611	6.92×
f5_l-d_kp_15_375	15	0.0472	0.1444	0.0972	3.06×
f6_l-d_kp_10_60	10	0.0341	0.0699	0.0358	2.05×
f7_l-d_kp_7_50	7	0.0141	0.0729	0.0588	5.17×
f8_l-d_kp_23_10000	23	0.0791	0.1850	0.1059	2.34×
f9_l-d_kp_5_80	5	0.0102	0.0542	0.0440	5.30×

f10_l-d_kp_20_879	20	0.0608	0.1311	0.0703	2.16×

Table c: Average runtime per instance across 10 seeds, with GA/ILS ratio

Overall, ILS records an average runtime of 0.0449 seconds per instance, compared to 0.1222 seconds for the GA. This means the GA is approximately 2.72 times slower on average across all runs.

5.3. Why ILS is Faster

The difference in runtime can be explained by the underlying structure of the algorithms.

- **GA overhead:** Each generation evaluates 100 individuals. Every evaluation uses the repair-and-fill operator, which requires sorting items by value-to-weight ratio. This is an $O(nlog\ n)$ operation performed 100 times per generation over 1,000 generations, resulting in 100,000 such operations per run. Additional steps such as selection, crossover, and mutation also contribute computational cost proportional to population size and chromosome length.
- **ILS overhead:** ILS performs 5,000 iterations, each operating on a single solution. The local search includes a 1-flip phase with $O(n)$ complexity and a 1-1 swap phase with worst-case $O(n^2)$ complexity. A greedy fill step introduces one $O(nlog\ n)$ operation per iteration. Since ILS maintains only one solution at a time, its total computational effort is significantly lower than that of the GA.

The runtime gap is largest for small instances such as f3 and f9. In these cases, the GA's fixed computational effort is excessive relative to the problem size. As the instance size increases, the gap narrows because the cost of ILS grows with the size of the neighbourhood.

5.4. Seed Stability

The GA exhibits relatively stable runtime across seeds, with a variation of approximately 1.17×. This is because a fixed number of generations and individuals determines its runtime. In contrast, ILS shows greater variability (about 1.37×) since its runtime depends on how quickly local search converges. Different seeds lead to different perturbations and search trajectories, affecting convergence speed. Despite this variability, solution quality remains unaffected, as all runs still reach the optimum.

5.5. Instance-Level Observations

- **f5_l-d_kp_15_375 (real-valued instance):** This is the only instance with non-integer values and weights. Both algorithms consistently find the optimal value of 481.0694, confirming correct handling of floating-point data. The variation in value-to-weight ratios enhances the effectiveness of the greedy initialisation in ILS.
- **f8_l-d_kp_23_10000 (near-equal items):** Items have very similar value-to-weight ratios, creating a subtle optimisation landscape. Both algorithms still identify the optimal solution of 9,767. ILS benefits from its swap neighbourhood, which allows fine adjustments between nearly equivalent items.
- **f3 and f4 (very small instances):** With only 16 possible subsets, the GA performs far more evaluations than necessary. This highlights the inefficiency of population-based approaches on trivial problem sizes.

5.6. Theoretical Context and Limitations

These findings highlight a fundamental trade-off in metaheuristic design. Genetic Algorithms excel at exploration in complex, multi-modal search spaces due to their population-based structure and recombination mechanisms (crossover). In contrast, Iterated Local Search focuses on exploitation through intensive local search and intelligent restarts.

For the small-scale instances used in this study (4–23 items), the problem landscape is simple enough that deep local search reliably finds the global optimum. Consequently, the additional overhead of maintaining a population of 100 individuals in the GA provides no advantage in solution quality and only increases runtime. This explains why ILS is significantly faster while achieving identical optimality.

However, this does not imply that ILS is universally superior. On larger, more complex instances (e.g., hundreds of items), the GA's ability to combine partial solutions via crossover may lead to better performance, consistent with the building-block hypothesis (Goldberg, 1989). Evaluating this would require a benchmark with significantly larger problem sizes.

Limitations of this study

1. **Benchmark scale:** With only 4 to 23 items, the instances are too small to discriminate between algorithms on solution quality. A comprehensive evaluation would include instances with 50, 100, 200, and 500 items, where neither algorithm consistently reaches the optimum and differences in best-found values become the primary comparison axis.

2. **Number of independent runs:** Although 10 seeds were used, a more rigorous study would employ at least 30 independent runs per configuration to properly characterise variance in solution quality (Barr et al., 1995).
3. **Fixed parameter settings:** The GA's population size, crossover rate, and mutation rate were not tuned per instance. The algorithm may therefore be over- or under-parameterised for specific instances, potentially affecting its efficiency.

These limitations do not invalidate the conclusions drawn for this benchmark, but they caution against over-generalising the results to all knapsack instances.

Conclusion

This study compared the performance of a Genetic Algorithm (GA) and Iterated Local Search (ILS) on ten 0/1 Knapsack benchmark instances using ten independent seeds. Both algorithms consistently achieved the known optimal solution on all 200 runs, resulting in identical solution quality and a 0.00% optimality gap across all instances. As a result, the Wilcoxon Signed-Rank Test on solution quality was degenerate, and no statistically significant difference in solution quality could be established.

Despite identical solution quality, the algorithms differed substantially in runtime performance. ILS was consistently faster than the GA on every instance and seed, with an average speed advantage of 2.72×. This performance gap is explained by the structural overhead of the GA, which evaluates large populations across many generations, compared to ILS, which operates on a single solution with focused local search.

These findings indicate that for small to medium-sized knapsack instances, ILS provides a more computationally efficient approach while maintaining optimal solution quality.
However, the benchmark instances used in this study are relatively low-dimensional, limiting the ability to distinguish algorithm performance in terms of solution quality. Future work should consider larger and more complex instances to evaluate how both algorithms scale and whether the GA's population-based search offers advantages in more challenging problem settings.

References

Bäck, T. (1996) Evolutionary Algorithms in Theory and Practice. Oxford: Oxford University Press.

Barr, R.S., Golden, B.L., Kelly, J.P., Resende, M.G.C. and Stewart, W.R. (1995) 'Designing and reporting on computational experiments with heuristic methods', Journal of Heuristics, 1(1), pp. 9–32. doi:10.1007/BF02430363.

De Jong, K.A. (1975) An Analysis of the Behaviour of a Class of Genetic Adaptive Systems. PhD thesis. University of Michigan.

Derrac, Joaquín & García, Salvador & Molina, Daniel & Herrera, Francisco. (2011). A practical tutorial on the use of nonprametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithm. Swarm and Evolutionary Computation. 1. 3-18. 10.1016/j.swevo.2011.02.002.

Conover, W.J. (1999) Practical Nonparametric Statistics. 3rd edn. New York: Wiley.

DeJong, K. (1975) An Analysis of the Behavior of a Class of Genetic Adaptive Systems. Ph.D. Dissertation, Department of Computer and Communication Sciences, University of Michigan, Ann Arbor.

Goldberg, D.E. (1989) Genetic Algorithms in Search, Optimization, and Machine Learning. Reading, MA: Addison-Wesley.

Goldberg, D.E. and Deb, K. (1991) A Comparative Analysis of Selection Schemes Used in Genetic Algorithms. In: Rawlins, G., Ed., Foundations of Genetic Algorithms, Morgan Kaufmann Publishers, San Mateo, CA, 69-93.

Holland, J.H. (1992) Adaptation in Natural and Artificial Systems. 2nd edn. Cambridge, MA: MIT Press.

Khuri, Sami & Bäck, Thomas & Heitkotter, Jorg. (1994). The Zero/One Multiple Knapsack Problem and Genetic Algorithms. 10.1145/326619.326694.

Lourenço, H.R., Martin, O.C., Stützle, T. (2003). Iterated Local Search. In: Glover, F., Kochenberger, G.A. (eds) Handbook of Metaheuristics. International Series in Operations Research & Management Science, vol 57. Springer, Boston, MA. https://doi.org/10.1007/0-306-48056-5_11

Martello, S. and Toth, P. (1990) Knapsack Problems: Algorithms and Computer Implementations. John Wiley & Sons Ltd., Chichester.

Michalewicz, Z. and Schoenauer, M. (1996) 'Evolutionary algorithms for constrained parameter optimization problems', Evolutionary Computation, 4(1), pp. 1–32. doi:10.1162/evco.1996.4.1.1.

Russell, S. and Norvig, P. (2022) Artificial Intelligence: A Modern Approach. 4th edn. Hoboken, NJ: Pearson.

Russell, S.J. & Norvig, P. (2020) Artificial Intelligence: A Modern Approach. 4th edn. Harlow: Pearson. (Global Edition)

Spears, W.M. and De Jong, K.A. (1991) 'On the virtues of parameterised uniform crossover', Proceedings of the 4th International Conference on Genetic Algorithms, pp. 230–236.

University of Pretoria (2026) COS314 Study Guide: Artificial Intelligence. Department of Computer Science, University of Pretoria.